Работа №1: pr4.ipynb

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("titanic.csv")
df.head(5)
```

Out[1]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/ O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05

In [2]:
```
y = df["Survived"]
X = df[["Pclass", "Age", "Sex", "Fare", "SibSp", "Parch"]].copy()

print("X:")
display(X.head())

print("y:")
display(y.head())
```

X:

	Pclass	Age	Sex	Fare	SibSp	Parch
0	3	22.0	male	7.2500	1	0
1	1	38.0	female	71.2833	1	0
2	3	26.0	female	7.9250	0	0
3	1	35.0	female	53.1000	1	0
4	3	35.0	male	8.0500	0	0

```
y:
0    0
1    1
2    1
3    1
4    0
Name: Survived, dtype: int64
```

In [3]:
```
print("Пропущенные значения:")
print(X.isnull().sum())
```

```
Пропущенные значения:
Pclass      0
Age       177
Sex         0
Fare        0
SibSp       0
Parch       0
dtype: int64
```

In [4]:
```
X.loc[:, "Age"] = X["Age"].fillna(X["Age"].median())

print("Пропуски после обработки Age:")
print(X.isnull().sum())
```

```
Пропуски после обработки Age:
Pclass    0
Age       0
Sex       0
Fare      0
SibSp     0
Parch     0
dtype: int64
```

In [5]:
```
X.loc[:, "Sex"] = X["Sex"].map({
    "male": 0,
    "female": 1
})

print("Подготовленные признаки:")
display(X.head())
```

```
Подготовленные признаки:
```

	Pclass	Age	Sex	Fare	SibSp	Parch
0	3	22.0	0	7.2500	1	0
1	1	38.0	1	71.2833	1	0
2	3	26.0	1	7.9250	0	0
3	1	35.0	1	53.1000	1	0
4	3	35.0	0	8.0500	0	0

In [6]:
```
print("Информация о X:")
X.info()

print("\nПропуски:")
print(X.isnull().sum())
```

```
Информация о X:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Pclass  891 non-null    int64
 1   Age     891 non-null    float64
 2   Sex     891 non-null    object
 3   Fare    891 non-null    float64
 4   SibSp   891 non-null    int64
 5   Parch   891 non-null    int64
dtypes: float64(2), int64(3), object(1)
memory usage: 41.9+ KB

Пропуски:
Pclass    0
Age       0
Sex       0
Fare      0
SibSp     0
Parch     0
dtype: int64
```

In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (712, 6)
X_test: (179, 6)
y_train: (712,)
y_test: (179,)
```

In [8]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("X_train_scaled:")
print(X_train_scaled[:5])

print("\nX_test_scaled:")
print(X_test_scaled[:5])
```

```
X_train_scaled:
[[-1.61413602  1.25364106 -0.7243102  -0.07868358 -0.47072241 -0.47934164]
 [-0.40055118 -0.47728355 -0.7243102  -0.37714494 -0.47072241 -0.47934164]
 [ 0.81303367  0.21508629 -0.7243102  -0.47486697 -0.47072241 -0.47934164]
 [ 0.81303367 -0.24649361 -0.7243102  -0.47623026  0.37992316 -0.47934164]
 [ 0.81303367 -1.78509326  1.38062393 -0.02524937  2.93185988  2.04874166]]

X_test_scaled:
[[ 0.81303367 -0.09263364 -0.7243102  -0.33390078  0.37992316  0.78470001]
 [-0.40055118  0.13815631 -0.7243102  -0.42528387 -0.47072241 -0.47934164]
 [ 0.81303367 -0.7080735  -0.7243102  -0.47486697 -0.47072241 -0.47934164]
 [-0.40055118 -1.78509326  1.38062393  0.00796649 -0.47072241  0.78470001]
 [ 0.81303367 -1.1696534   1.38062393 -0.41100201  0.37992316 -0.47934164]]
```

In [9]:
```python
print("Размер X_train:", X_train.shape)
print("Размер X_test:", X_test.shape)
print("Размер y_train:", y_train.shape)
print("Размер y_test:", y_test.shape)
print("Размер X_train_scaled:", X_train_scaled.shape)
print("Размер X_test_scaled:", X_test_scaled.shape)

print("\nГотово: данные очищены, Sex преобразован в числа,")
print("выборки разделены в соотношении 80/20 и выполнено масштабирование.
```

```
Размер X_train: (712, 6)
Размер X_test: (179, 6)
Размер y_train: (712,)
Размер y_test: (179,)
Размер X_train_scaled: (712, 6)
Размер X_test_scaled: (179, 6)

Готово: данные очищены, Sex преобразован в числа,
выборки разделены в соотношении 80/20 и выполнено масштабирование.
```

Работа №2: pr5.ipynb

In [1]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("titanic.csv")
df.head()
```

Out[1]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/ O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05

In [2]:

```
X = df[["Pclass", "Age", "Sex", "Fare", "SibSp", "Parch"]].copy()
y = df["Survived"]

print("Пропуски до обработки:")
print(X.isnull().sum())

X.loc[:, "Age"] = X["Age"].fillna(X["Age"].median())
X.loc[:, "Sex"] = X["Sex"].map({"male": 0, "female": 1})

print("\nПропуски после обработки:")
print(X.isnull().sum())
print("\nТипы данных:")
print(X.dtypes)
print("Пропусков в X:", int(X.isnull().sum().sum()))
```

```
Пропуски до обработки:
Pclass      0
Age       177
Sex         0
Fare        0
SibSp       0
Parch       0
dtype: int64

Пропуски после обработки:
Pclass    0
Age       0
Sex       0
Fare      0
SibSp     0
Parch     0
dtype: int64

Типы данных:
Pclass      int64
Age       float64
Sex        object
Fare      float64
SibSp       int64
Parch       int64
dtype: object
Пропусков в X: 0
```

In [3]:
```python
data_corr = X.copy()
data_corr["Survived"] = y

print("Столбцы таблицы:")
print(data_corr.columns.tolist())
print("\nПервые строки:")
display(data_corr.head())
print("\nТипы данных:")
print(data_corr.dtypes)
print("\nКоличество пропусков:")
print(data_corr.isnull().sum())
```

```
Столбцы таблицы:
['Pclass', 'Age', 'Sex', 'Fare', 'SibSp', 'Parch', 'Survived']

Первые строки:
```

	Pclass	Age	Sex	Fare	SibSp	Parch	Survived
0	3	22.0	0	7.2500	1	0	0
1	1	38.0	1	71.2833	1	0	1
2	3	26.0	1	7.9250	0	0	1
3	1	35.0	1	53.1000	1	0	1
4	3	35.0	0	8.0500	0	0	0

```
Типы данных:
Pclass        int64
Age         float64
Sex          object
Fare        float64
SibSp         int64
Parch         int64
Survived      int64
dtype: object

Количество пропусков:
Pclass      0
Age         0
Sex         0
Fare        0
SibSp       0
Parch       0
Survived    0
dtype: int64
```

In [4]:
```
corr_matrix = data_corr.corr(numeric_only=True).round(2)
print("Корреляционная матрица:")
display(corr_matrix)

survived_corr = corr_matrix["Survived"].drop("Survived")
positive_feature = survived_corr.idxmax()
negative_feature = survived_corr.idxmin()

print("Самая сильная положительная корреляция с Survived:")
print(positive_feature, survived_corr[positive_feature])
print("\nСамая сильная отрицательная корреляция с Survived:")
print(negative_feature, survived_corr[negative_feature])
```

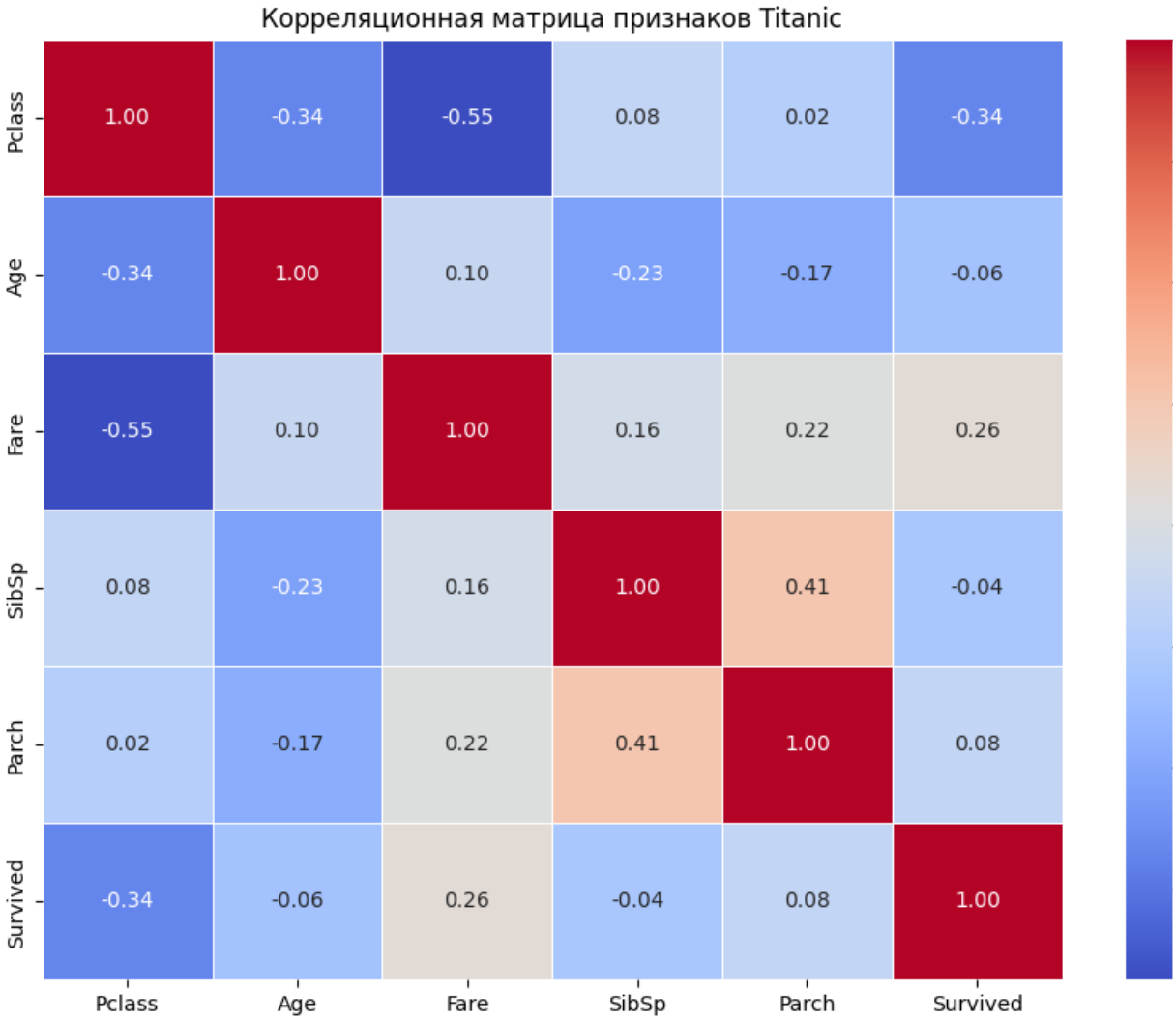
```
Корреляционная матрица:
```

	Pclass	Age	Fare	SibSp	Parch	Survived
Pclass	1.00	-0.34	-0.55	0.08	0.02	-0.34
Age	-0.34	1.00	0.10	-0.23	-0.17	-0.06
Fare	-0.55	0.10	1.00	0.16	0.22	0.26
SibSp	0.08	-0.23	0.16	1.00	0.41	-0.04
Parch	0.02	-0.17	0.22	0.41	1.00	0.08
Survived	-0.34	-0.06	0.26	-0.04	0.08	1.00

```
Самая сильная положительная корреляция с Survived:
Fare 0.26

Самая сильная отрицательная корреляция с Survived:
Pclass -0.34
```

In [5]:
```
plt.figure(figsize=(9, 7))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidt
plt.title("Корреляционная матрица признаков Titanic")
plt.tight_layout()
plt.show()
```



In [6]:
```
print("Краткий вывод:")
print(f"Наиболее сильная положительная связь с Survived: {positive_featur
print(f"Наиболее сильная отрицательная связь с Survived: {negative_featur
print("Для предсказания могут быть полезны Pclass, Sex и Fare.")
print("Выбирать признаки только по корреляции нельзя: нужно проверять кач
print("Корреляция не доказывает причинно-следственную связь.")
```

```
Краткий вывод:
Наиболее сильная положительная связь с Survived: Fare (r = 0.26).
Наиболее сильная отрицательная связь с Survived: Pclass (r = -0.34).
Для предсказания могут быть полезны Pclass, Sex и Fare.
Выбирать признаки только по корреляции нельзя: нужно проверять качество модели
тестовых данных.
Корреляция не доказывает причинно-следственную связь.
```

Работа №3: pr6Titanic.ipynb

Open in Colab

In [7]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)

df = pd.read_csv("titanic.csv")
print(df.head())
print("Размер исходных данных:", df.shape)
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
Размер исходных данных: (891, 12)
```

1. Подготовка данных

In [8]:

```
features = ["Pclass", "Age", "Sex", "Fare", "SibSp", "Parch"]
X = df[features].copy()
y = df["Survived"]

X["Age"] = X["Age"].fillna(X["Age"].median())
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})

print("Пропуски в признаках:")
print(X.isnull().sum())
print("\nТипы данных:")
print(X.dtypes)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nРазмер X_train:", X_train.shape)
print("Размер X_test:", X_test.shape)
print("Размер y_train:", y_train.shape)
print("Размер y_test:", y_test.shape)
```

```
Пропуски в признаках:
Pclass    0
Age       0
Sex       0
Fare      0
SibSp     0
Parch     0
dtype: int64

Типы данных:
Pclass      int64
Age       float64
Sex         int64
Fare      float64
SibSp       int64
Parch       int64
dtype: object

Размер X_train: (712, 6)
Размер X_test: (179, 6)
Размер y_train: (712,)
Размер y_test: (179,)
```

Ответ на вопрос: тестовая выборка нужна для независимой оценки качества на данных, к модель не использовала при обучении. `scaler.fit_transform()` применяется только обучающей выборке, чтобы параметры масштабирования не подсматривали в тестовые да тестовой выборке применяется только `transform()`.

2. Логистическая регрессия и предсказания

In [9]:
```python
model_lr = LogisticRegression(max_iter=1000)
model_lr.fit(X_train_scaled, y_train)

y_pred_lr = model_lr.predict(X_test_scaled)
print("Первые предсказания:", y_pred_lr[:10])
print("Правильные ответы:  ", y_test.iloc[:10].to_list())
```

```
Первые предсказания: [0 0 0 0 1 0 1 0 0 0]
Правильные ответы:   [0, 0, 1, 0, 1, 1, 1, 0, 0, 0]
```

Ответ: `X_train_scaled` — признаки обучающей выборки, `y_train` — правильные кла обучения. Тестовые данные не используются при обучении, чтобы оценка была честной. З `0` означает, что пассажир не выжил, `1` — что выжил. Сравните две строки выше: совпад правильные предсказания, различия — ошибки.

3. Метрики логистической регрессии

In [10]:
```python
def show_metrics(name, y_true, y_pred):
    print(f"--- {name} ---")
    print("Accuracy:", round(accuracy_score(y_true, y_pred), 4))
    print("Precision:", round(precision_score(y_true, y_pred, zero_divisi
    print("Recall:", round(recall_score(y_true, y_pred, zero_division=0),
    print("F1:", round(f1_score(y_true, y_pred, zero_division=0), 4))
    print("Матрица ошибок [[TN, FP], [FN, TP]]:")
    print(confusion_matrix(y_true, y_pred, labels=[0, 1]))
```

```
    print(classification_report(y_true, y_pred, zero_division=0))

show_metrics("Логистическая регрессия", y_test, y_pred_lr)
```

```
--- Логистическая регрессия ---
Accuracy: 0.8045
Precision: 0.7656
Recall: 0.7101
F1: 0.7368
Матрица ошибок [[TN, FP], [FN, TP]]:
[[95 15]
 [20 49]]
              precision    recall  f1-score   support

           0       0.83      0.86      0.84       110
           1       0.77      0.71      0.74        69

    accuracy                           0.80       179
   macro avg       0.80      0.79      0.79       179
weighted avg       0.80      0.80      0.80       179
```

Ответы: Accuracy — доля правильных предсказаний. Recall для класса `1` показывает, ка
долю реально выживших модель обнаружила. Ложноположительный результат (FP) — мод
предсказала выживание, хотя пассажир не выжил. Ложноотрицательный (FN) — модель
предсказала, что пассажир не выжил, хотя он выжил.

4. Дерево решений и сравнение моделей

In [11]:
```
model_tree = DecisionTreeClassifier(max_depth=4, random_state=42)
model_tree.fit(X_train, y_train)
y_pred_tree = model_tree.predict(X_test)

show_metrics("Дерево решений", y_test, y_pred_tree)

comparison = pd.DataFrame({
    "Модель": ["Логистическая регрессия", "Дерево решений"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_lr),
        accuracy_score(y_test, y_pred_tree)
    ],
    "Precision": [
        precision_score(y_test, y_pred_lr, zero_division=0),
        precision_score(y_test, y_pred_tree, zero_division=0)
    ],
    "Recall": [
        recall_score(y_test, y_pred_lr, zero_division=0),
        recall_score(y_test, y_pred_tree, zero_division=0)
    ],
    "F1": [
        f1_score(y_test, y_pred_lr, zero_division=0),
        f1_score(y_test, y_pred_tree, zero_division=0)
    ]
})
comparison.round(4)
```

```
--- Дерево решений ---
Accuracy: 0.7821
Precision: 0.875
Recall: 0.5072
F1: 0.6422
Матрица ошибок [[TN, FP], [FN, TP]]:
[[105   5]
 [ 34  35]]
              precision    recall  f1-score   support

           0       0.76      0.95      0.84       110
           1       0.88      0.51      0.64        69

    accuracy                           0.78       179
   macro avg       0.82      0.73      0.74       179
weighted avg       0.80      0.78      0.77       179
```

Out[11]:

	Модель	Accuracy	Precision	Recall	F1
0	Логистическая регрессия	0.8045	0.7656	0.7101	0.7368
1	Дерево решений	0.7821	0.8750	0.5072	0.6422

Ответ: обе модели используют одни и те же `X_train`, `X_test`, `y_train` и `y_test`, и сравнение корректное. По таблице выше определите, у какой модели выше Accuracy и F1; вписывайте числа заранее — они зависят от конкретного файла `titanic.csv`.

5. Связь с корреляционным анализом из ПР № 5

In [12]:
```
corr_df = df[["Survived", "Pclass", "Age", "Fare", "SibSp", "Parch"]].cop
corr_df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
corr = corr_df.corr(numeric_only=True)["Survived"].drop("Survived").sort_
print(corr)
print("\nНаиболее сильная положительная корреляция:", corr.idxmax(), roun
print("Наиболее сильная отрицательная корреляция:", corr.idxmin(), round(
```

```
Pclass   -0.338481
Age      -0.077221
SibSp    -0.035322
Parch     0.081629
Fare      0.257307
Sex       0.543351
Name: Survived, dtype: float64

Наиболее сильная положительная корреляция: Sex 0.5434
Наиболее сильная отрицательная корреляция: Pclass -0.3385
```

Вывод

В работе подготовлены признаки пассажиров Titanic, данные разделены на обучающую и т выборки, обучены две модели бинарной классификации и рассчитаны Accuracy, Precision, F1 и матрицы ошибок. Фактические значения метрик и названия признаков с максимально минимальной корреляцией нужно взять из вывода ячеек после запуска ноутбука на `titanic.csv`.